ARM PrimeCell

RTC (PL031)

Design Manual

ARM

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	By	Change
0.1	August 6, 2001	Michael Popoola	Initial draft of design manual: ARM PrimeCell (PL031) Design Manual.
0.2,0.3	August 10, 2001	Michael Popoola	Added more text on Functional Verification details
1.0	August 22, 2001	Michael Popoola	First version includes changes from document review

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
[1]	ARM DDI0224A-04	ARM PrimeCell RTC (PL031) Technical Reference Manual
[2]	SC058-INTC-00006	ARM PrimeCell RTC (PL031) Integration Manual
[3]	ARM DUI 0006	AMBA Compliance Test Suite User Guide
[4]	ARM DDI 0138	Example AMBA System Technical Reference Manual
[5]	ARM DUI 0092	Example AMBA System User Guide
[6]	P008405US	Application Papers For an Interface Mechanism
[7]	SCB00 DESR 0001 B	AMBA Peripherals Design Manual

1.3 Terms and abbreviations

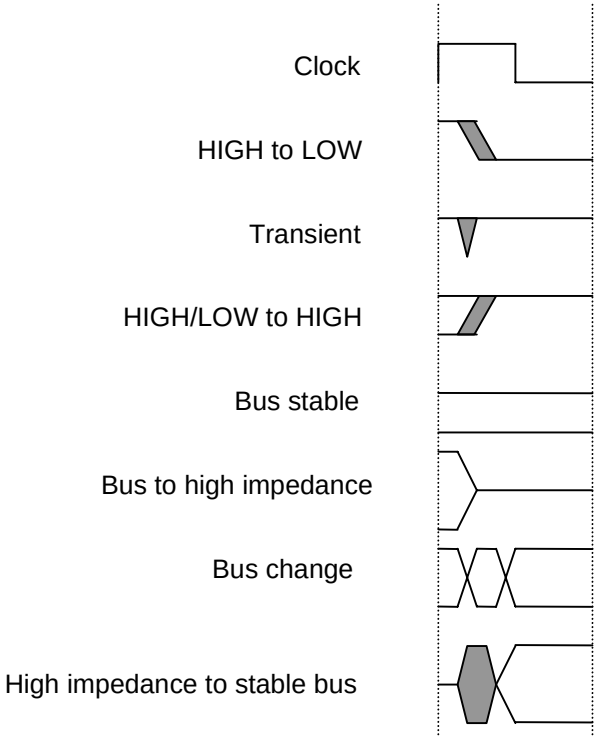
This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTC	Real Time Clock
RTL	Register Transfer Level
SoC	System On-Chip
TIC	Test Interface Controller

TRM Technical Reference Manual

1.4 Timing diagram conventions

This manual contains one or more timing diagrams. The following key explains the components used in these diagrams. Any variations are clearly labelled when they occur. Therefore, no additional meaning should be attached unless specifically stated.



2 SCOPE

This document describes the design and implementation of the ARM PrimeCell RTC (Real Time Clock).

Section 4 presents an overview of the RTC design, describing the functional partitioning of the RTC and the signal interconnections. Section 5 presents detailed descriptions on the RTC design.

3 INTRODUCTION

The RTC is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell RTC (PL031) Technical Reference Manual [Ref.1]. The ARM PrimeCell RTC (PL031) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of BusTalk, TICTalk test vectors for functional verification and integration testing.

4 RTC DESIGN OVERVIEW

4.1 Functional Description

The ARM PrimeCell Real Time Clock (RTC) can be used to provide a basic alarm function or long time base counter. This is achieved by generating an interrupt signal after counting for a programmed number of cycles of a real-time clock input. Counting in one-second intervals is achieved by use of a 1HZ clock input to the PrimeCell RTC. A block diagram is shown in **Figure 4-1 ARM PrimeCell RTC Block Diagram.**

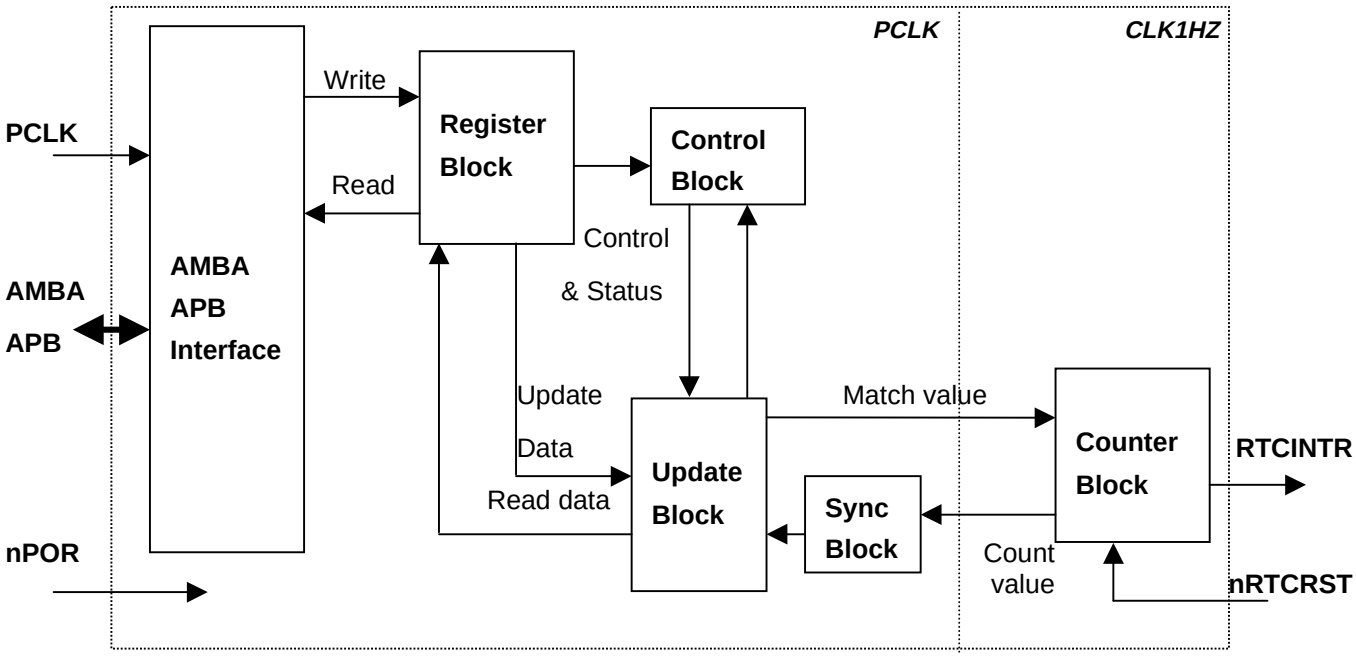


Figure 4-1 ARM PrimeCell RTC Block Diagram

The ARM PrimeCell uses a patented interface [Ref.6] mechanism to reduce latency between load and update times of some real time clock value. And the design ensures that the PrimeCell RTC can generated interrupts even when PCLK is not running, which can happen during a device shutdown, by generating the interrupt in the CLK1HZ domain, and retaining match and count values required for generating the interrupt.

4.2 Signal Description

The following tables summarise the RTC signal list. Table 4-1 lists the APB interface signals while Table 4-2 lists the block specific non-APB signals.

Table 4-1 APB Signal descriptions

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Bus Clock.
PRESETn	IN	Reset Controller	Bus Reset (active low).
PENABLE	IN	APB Bridge	This signal is used to time all accesses on the peripheral bus
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PADDR[11:2]	IN	APB Bridge	This is part of the peripheral address bus, which is used for decoding the internal address space.
PWDATA[31:0]	IN	APB Bridge	Data is written into the peripheral on this bus.
PRDATA[31:0]	OUT	APB Bridge	Data is read out of the peripheral via this bus.

Table 4-2 Block Specific signal descriptions

Signal Name	Type	Source / Destination	Description
CLK1HZ	IN	Clock Generator	1Hz clock input. This is the signal that clocks the counter during normal operation.
nPOR	IN	Reset controller	PrimeCell RTC power-on reset signal to Match and Offset registers. They need to retain their values over a bus reset.
nRTCRST	IN	Reset controller	PrimeCell RTC reset signal to CLK1HZ domain, active LOW. The reset controller must use PRESETn to assert nRTCRST asynchronously but negate it synchronously to CLK1HZ.
RTCINTR	OUT	Interrupt Controller	Interrupt signal to the Interrupt module. When HIGH, this signal indicates that a match has occurred between the RTC value and the match register.
SCANENABLE	IN	Test Controller	RTC scan enable signal for both clock domains.
SCANINPCLK	IN	Test Controller	RTC input scan signal for the PCLK domain.
SCANINCLK1HZ	IN	Test Controller	RTC input scan signal for the CLK1HZ domain.
SCANOUTPCLK	OUT	Test Controller	RTC output scan signal for the PCLK domain.
SCANOUTCLK1HZ	OUT	Test Controller	RTC output scan signal for the CLK1HZ domain.

4.3 Programmer's Model

4.3.1 Functional Mode Registers

Table 4-3 RTC Register summary

Address	Type	Width	Reset value	Name	Description
RTC Base + 0x000	Read	32	0x00000000	RTCDR	Data Register.
RTC Base + 0x004	Read/write	32	0x00000000	RTCMR	Match Register.
RTC Base + 0x008	Read/write	32	0x00000000	RTCLR	Load register.
RTC Base + 0x00C	Read/write	1	0x00000000	RTCCR	Control register.
RTC Base + 0x010	Read/write	1	0x00000000	RTCIMSC	Interrupt mask set and clear register
RTC Base + 0x014	Read	1	0x00000000	RTCRIS	Raw interrupt status register.
RTC Base + 0x018	Read	1	0x00000000	RTCMIS	Masked interrupt status register.
RTC Base + 0x01C	Write	1	0x00000000	RTCICR	Interrupt clear register.
RTC Base + 0x020–07C	-	-	-	-	Reserved.
RTC Base + 0x080–090	-	-	-	-	Reserved (for test purposes).
RTC Base +0x094–FCC	-	-	-	-	Reserved.
RTC Base +0xFD0–FDC	-	-	-	-	Reserved for future ID expansion
RTC Base + 0xFE0	Read	8	0x31	RTCPeriphID0	Peripheral identification register bits 7:0
RTC Base + 0xFE4	Read	8	0x10	RTCPeriphID1	Peripheral identification register bits 15:8
RTC Base + 0xFE8	Read	8	0x04	RTCPeriphID2	Peripheral identification register bits 23:16
RTC Base + 0xFEC	Read	8	0x00	RTCPeriphID3	Peripheral identification register bits 31:24
RTC Base + 0xFF0	Read	8	0x0D	RTCPCellID0	PrimeCell identification register bits 7:0
RTC Base + 0xFF4	Read	8	0xF0	RTCPCellID1	PrimeCell identification register bits 15:8
RTC Base + 0xFF8	Read	8	0x05	RTCPCellID2	PrimeCell identification register bits 23:16
RTC Base + 0xFFC	Read	8	0xB1	RTCPCellID3	PrimeCell identification register bits 31:24

Please see the PrimeCell RTC (PL031) TRM [Ref.1] for a detailed description of each register.

4.3.2 Test Mode Registers

Table 4-4 Test Mode Registers

Address	Type	Width	Reset value	Name	Description
RTC Base + 0x80	Read/write	3	0x00000000	RTCITCR	Integration Test Control register.
RTC Base + 0x84	Read/write	0	0x00000000	RTCITIP	Integration test input read/set register
RTC Base + 0x88	Read/write	1	0x00000000	RTCITOP	Integration test output read/set register
RTC Base + 0x8C	Read/write	32	0x00000000	RTCTOFFSET	Test Offset register
RTC Base + 0x90	Read/write	32	0x00000000	RTCTCOUNT	Test Count register

Please see PrimeCell RTC (PL031) TRM for a detailed description of each register.

4.3.3 Register Memory Map

Table 4-5 Register Memory Map

Address	Read Location	Write Location
RTC Base + 0x000	RTCDR	-
RTC Base + 0x004	RTCMR	RTCMR
RTC Base + 0x008	RTCLR	RTCLR
RTC Base + 0x00C	RTCCR	RTCCR
RTC Base + 0x010	RTCIMSC	RTCIMSC
RTC Base + 0x014	RTCRIS	-
RTC Base + 0x018	RTCMIS	-
RTC Base + 0x01C	-	RTCICR
RTC Base + 0x020–07C	Reserved	Reserved
RTC Base + 0x080	RTCITCR	RTCITCR
RTC Base + 0x084	RTCITIP	RTCITIP
RTC Base + 0x088	RTCITOP	RTCITOP
RTC Base + 0x08C	RTCTOFFSET	RTCTOFFSET

RTC Base + 0x090	RTCTCOUNT	RTCTCOUNT

Table 4-5 Register Memory Map (continued)

RTC Base +0x094–FCC	Reserved	Reserved
RTC Base +0xFD0–FDC	Reserved for future ID expansion	Reserved for future ID expansion
RTC Base + 0xFE0	RTCPeriphID0	-
RTC Base + 0xFE4	RTCPeriphID1	-
RTC Base + 0xFE8	RTCPeriphID2	-
RTC Base + 0xFEC	RTCPeriphID3	-
RTC Base + 0xFF0	RTCPCellID0	-
RTC Base + 0xFF4	RTCPCellID1	-
RTC Base + 0xFF8	RTCPCellID2	-
RTC Base + 0xFFC	RTCPCellID3	-

4.4 Block Partitioning

The sub blocks in this design are listed below.

- APB interface, register block, and test logic block
- Asynchronous Interrupt logic
- Update logic
- Synchronisers
- Counter block
- Control block
- Revision Designator

4.4.1 APB Interface

The APB interface comprises an input and output module. This interface deals with reads and writes to registers from the APB. It also contains the register and test logic blocks. They are all integrated into one block to reduce signal movement across blocks for generation of normal-mode and test registers.

The test logic contains the test mode registers in the RTC, and the logic for the integration tests. The integration test allows the RTCINTR output to be written to via the Integration Test Output Register (RTCITOP) under the control of the integration test enable signal (ITEN).

A simplified diagram of the APB interface block is shown in **Figure 4-2 APB Interface, Register and Test logic Block diagram.**

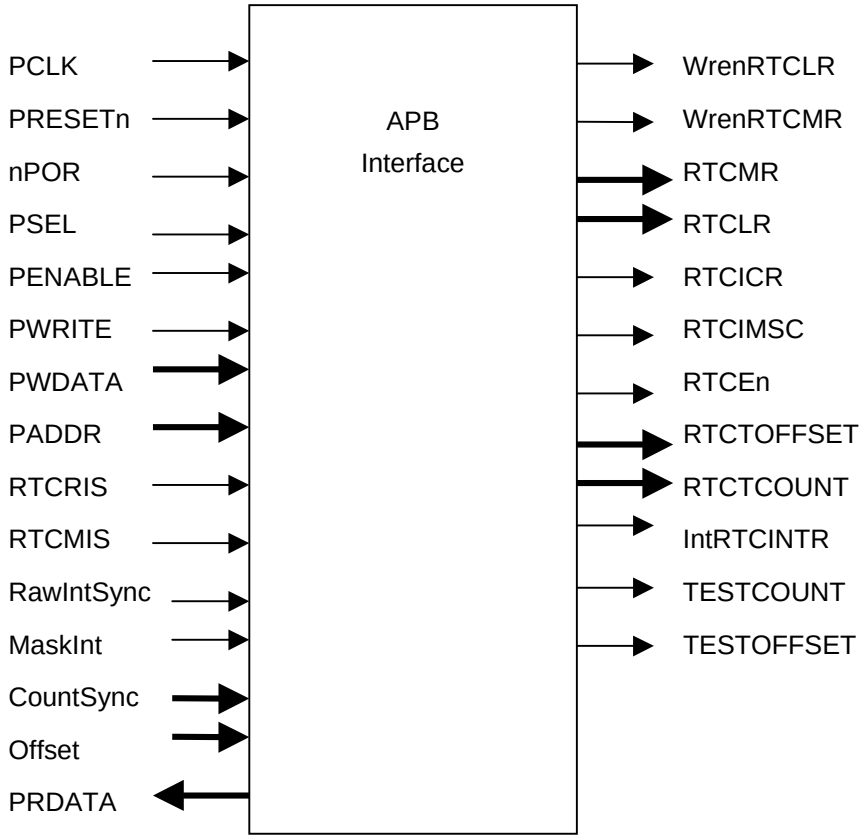


Figure 4-2 APB Interface, Register and Test logic Block diagram

Input Module

This module performs the following functions:

- It provides the APB write interface. It generates decodes for writes to all registers in the device. Each register addressable from the APB is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode and PENABLE, PSEL and PWRITE signals.
- It contains all the normal-mode PrimeCell RTC registers.
- It contains the test mode PrimeCell RTC registers. The sole non-AMBA intra-chip output RTCINTR is multiplexed with its corresponding integration test register bit.

The block diagram in **Figure 4-3 Input module** illustrates the structure of the input module.

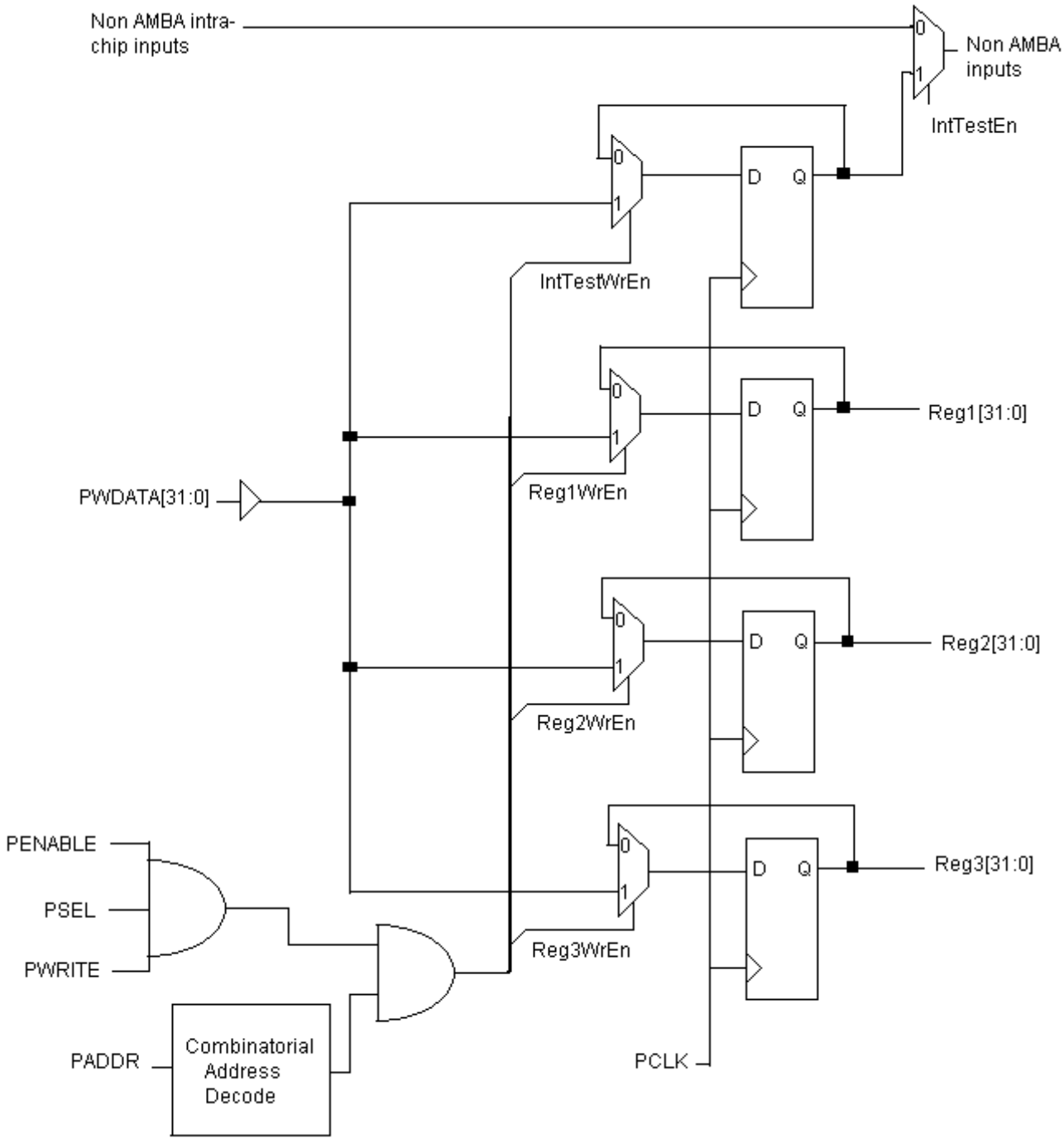


Figure 4-3 Input module

Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read data bus. The output of the multiplexer is fed to a bank of flip-flops clocked by the system PCLK. A combinatorial address decode is used to select which of the output sources are driven onto the bus. Non-AMBA outputs such as the interrupt signal is made observable through APB read accesses – in this case through the RTC raw interrupt status and masked interrupt status registers (RTCRIS and RTCMIS). The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no

restrictions placed on the type of external bus connection method used for the data bus on the APB. A block diagram of the output module is shown in **Figure 4-4 Output Module.**

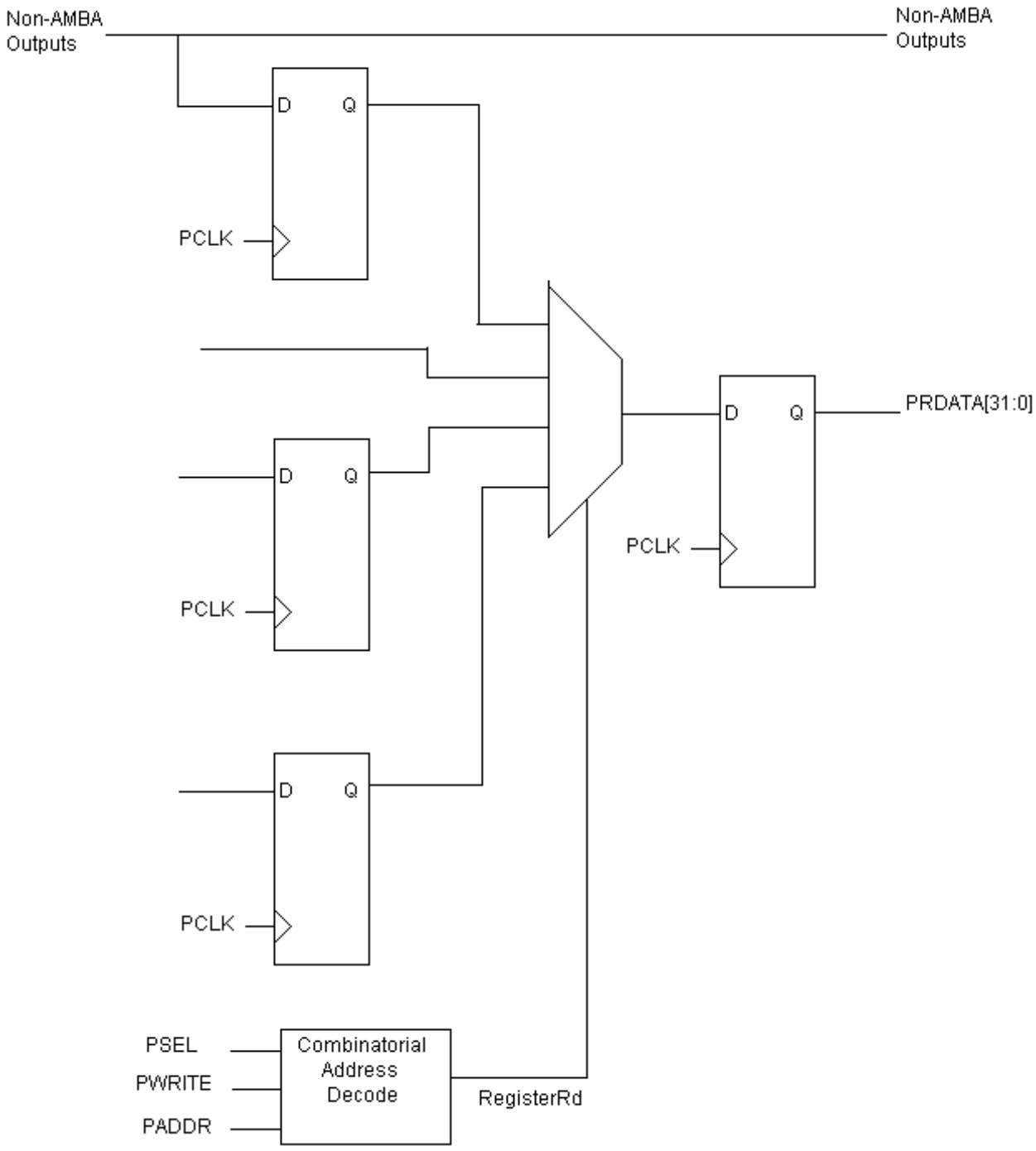


Figure 4-4 Output module

4.4.2 Control Block

The control block generates the control signal, IntClear.

Figure 4-5 shows the block diagram of the Control block.

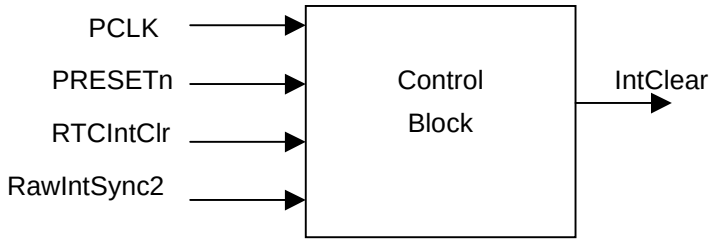


Figure 4-5 Control Block

4.4.3 Counter Block

The 32 bit real time clock (RTC) counter is located in the counter block in the **CLK1HZ** clock domain. It is 0x00000001 on reset, and counts up to 0xFFFFFFFF where it wraps round to 0x00000000 and continues incrementing. During normal operation, it is free running and is not updated at any time except when reset to 0x00000001 by the **CLK1HZ** domain reset signal, nRTCRST. The reset signal should only be used during a system hard reset.

During testing, the count register is accessible to be written to and read from. When the TESTCOUNT bit in the RTCITCR register is set, data written to the RTCTCOUNT register is loaded into the counter. When TESTCOUNT is cleared, the counter resumes normal operation. This test-mode is included only for block verification and should not be used in real hardware.

Figure 4-6 shows the block diagram of the Counter block.

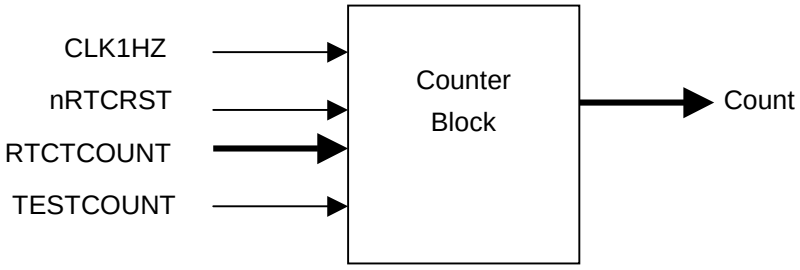


Figure 4-6 Counter Block

4.4.4 Interrupt Block

This block contains the asynchronous combinatorial interrupt logic. The block diagram of this block is shown in Figure 4-7 Asynchronous Interrupt Generation.

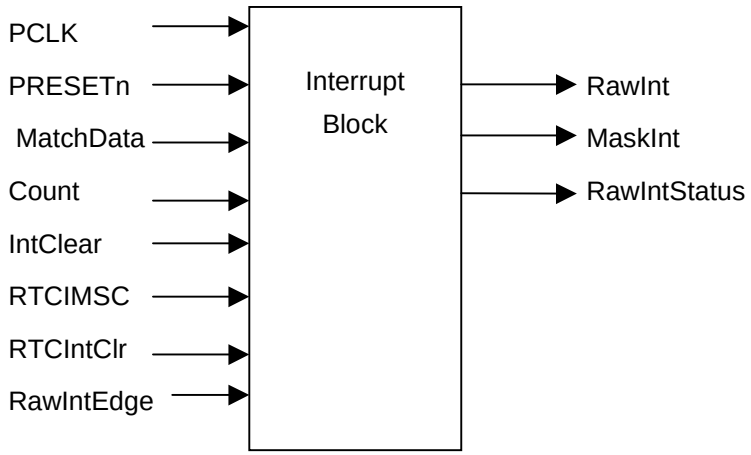


Figure 4-7 Interrupt Block

The RTC interrupt generated in the CLK1HZ domain is treated as being asynchronous to the final processor clock domain, PCLK. It is asserted in the CLK1HZ domain when the Count value is equal to the equivalent match register value (MatchData), but synchronously removed by writing to the interrupt clear register (RTCICR) which resides in the PCLK domain. MatchData is calculated by applying the offset data generated in the Update block to data in the match register (RTCMR). This is discussed in more detail in **Section 5-7 Update logic block** on page 23**.**

4.4.5 Synchronisers

Synchronisation is required for any signal crossing between the two clock domains. The signals synchronised in this block are the raw interrupt, masked interrupt and count register value as shown in the diagram below.

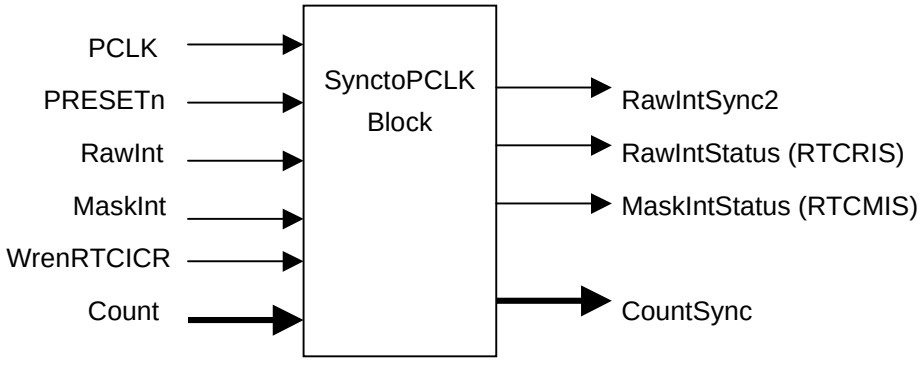


Figure 4-8 SynctoPCLK Block

4.4.6 Revision Designator block

The block diagram is shown in Figure 4-9 Revision Designator block. It contains a single AND gate to be used as a placeholder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These “TieOffs” can be identified during layout and re-wired to “VDD” or “VSS” if needed.

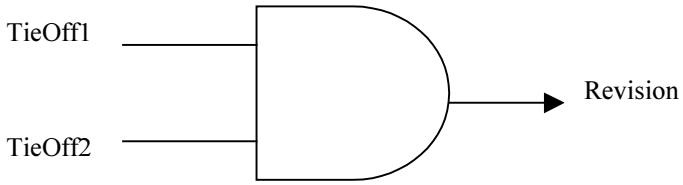


Figure 4-9 Revision Designator block

5 RTC IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The design hierarchy in the RTC is shown in Figure 5-1. A short description about the functionality of each of the sub-blocks is also given within brackets.

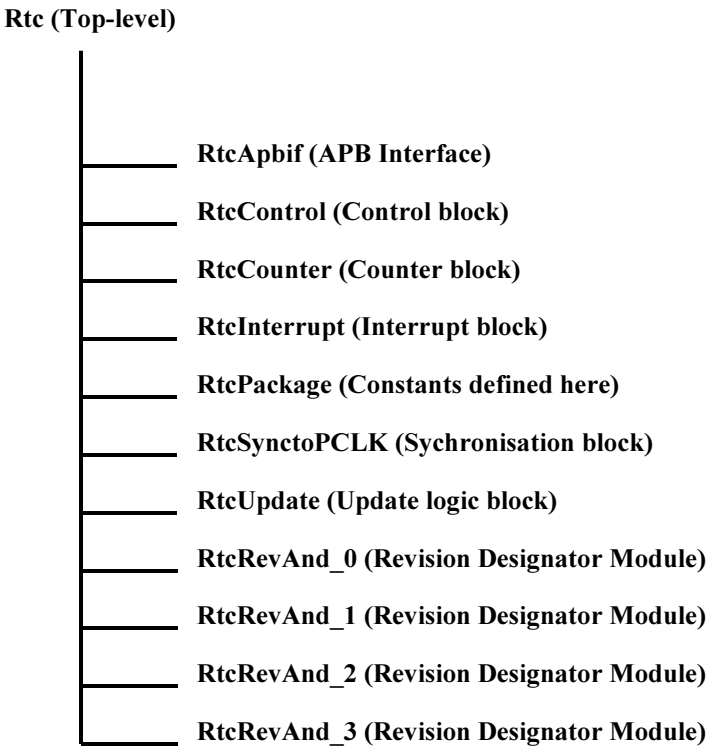


Figure 5-1 RTC Design Hierarchy

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the RTC.

5.2 RTC Top Level Block (Rtc)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the RTC.

5.3 RTC APB Interface (RtcApbif)

The APB interface generates decodes for writes into the registers in the address space of the RTC. It contains the register and test blocks and also contains the read interface for the RTC.

An internal write data bus, PWDATAIn[15:0], is generated by gating the internal write data bus input to the RTC, PWDAT[15:0], with PSEL and PWRITE. This prevents the data bus internal to the RTC from toggling when the device is not being written into or when it is not selected.

5.3.1 Write Interface

The write strobe for each register is generated with a decode of PSEL, PWRITE, PADDR[11:2] and PENABLE. Data is clocked in to the selected register on the rising edge of PCLK on which the register Write enable is asserted.

5.3.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the data bus. These flip-flops are clocked by the rising edge of PCLK. Select inputs to the output multiplexer are generated with a combinatorial decode of PWRITE, PADDR, and PSEL signals. Since these signals are stable from the rising edge of PCLK, almost one full PCLK period is available for the output of the multiplexer to stabilise before data is clocked in on the next rising edge of PCLK. Data is sampled by the APB bridge on the next rising edge of PCLK on which PENABLE is de-asserted.

The external data bus from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexer bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus when it is not selected. In the RTC, the output multiplexer is left such that, when the device is not selected, it drives zeroes onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

5.3.3 Register block

The register block contains the normal mode registers in the RTC. It also generates the signal that clears the interrupt. The clear signal is generated when a ‘1’ is written into the RTCICR register.

5.3.4 Test logic

The test block performs the following functions:

- Test mode control via the RTCITCR register.
- Integration testing using RTCITIP and RTCITOP read/set registers.
- Test read/write access of the Count and Offset registers respectively.

The RTCITCR integration test control register contains three programmable bits:

- ITEN, the integration test enable. When set, the RTC is placed into integration test mode. This mode allows point to point connections to be checked between other modules when the RTC is instantiated within a system. Two registers, RTCITIP (Integration Test Input) and RTCITO (Integration Test Output) are used to implement this test mode.
- TESTCOUNT, the test-count enable bit. When set, data can be read from or written to the count register for test purposes.
- TESTOFFSET, the test offset enable bit. When set, data can be read from or written to the offset register for test purposes.

5.3.4.1 Integration Test Logic – Intra-chip output

Use this test for the RTCINTR output. When you run integration tests with the PrimeCell RTC in a standalone test set-up:

- Write a 1 to the ITEN bit in the integration test control register. This selects the test path from the RTCITOP register bit to the intra-chip output signals.
- Write a 1 and then a 0 to the RTCITOP register bit, and read the same register bit to verify that the value written is read out.

When you run integration tests with the PrimeCell RTC as part of an integrated system:

- Write a 1 to the ITEN bit in the integration test control register. This selects the test path from the RTCITOP register bit to the intra-chip output signals.
- Write a 1 and then a 0 to the RTCITOP register bit to toggle the signal connections between the interrupt controller and the PrimeCell RTC. Read from the internal test registers of the interrupt controller to verify that the value written into the RTCITOP register bit is read out through the PrimeCell RTC.

Figure 5-2 Output integration test harness, intra-chip outputs explains the implementation details of the output integration test harness for intra-chip outputs.

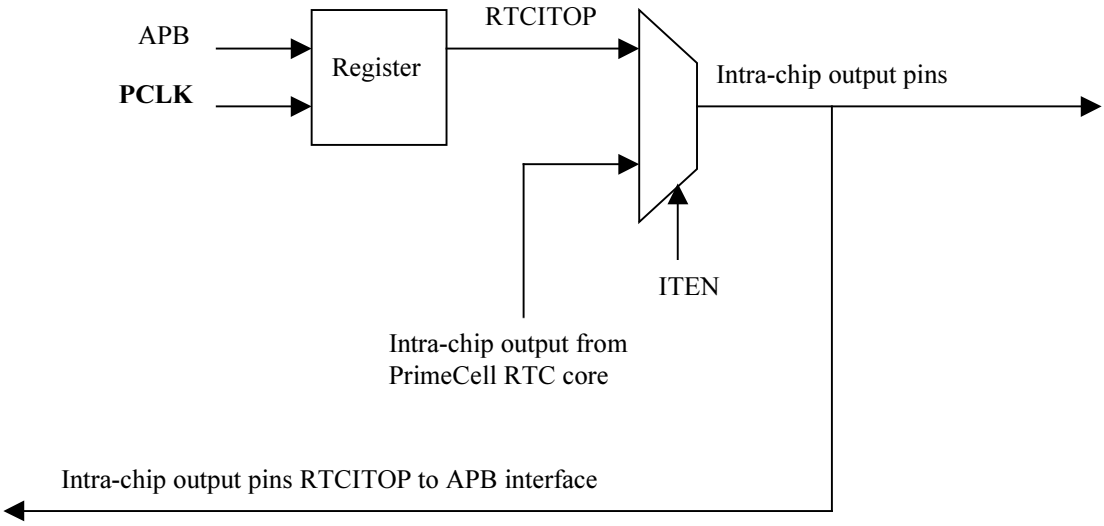


Figure 5-2 Output integration test harness, intra-chip output

5.4 RTC Control Block (RtcControl)

The control block generates the IntClear signal.

Figure 5-3 IntClear signal generation illustrates the generation of the IntClear signal. The IntClear signal is used in the interrupt block for the process of generating the masked interrupt signal.

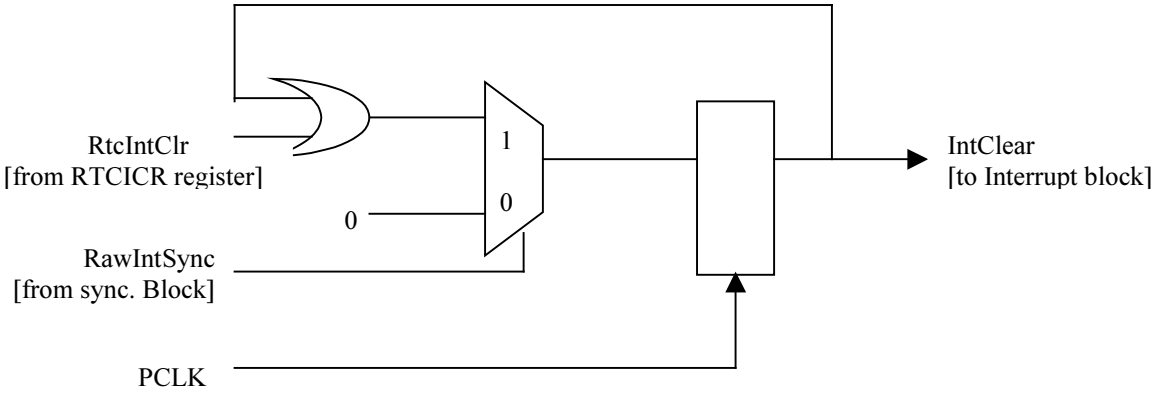


Figure 5-3 IntClear Signal Generation

IntClear is asserted if there has been a write to the RTC interrupt clear register (RTCICR) while there is a synchronised raw interrupt (indicated by the synchronised raw interrupt signal, RawIntSync). And it remains asserted until the raw interrupt line goes LOW. It is used in the interrupt logic block to determine the assertion of the masked interrupt and hence the RTC interrupt output, RTCINTR. See **Section 5-5 Interrupt Logic Block** on Page 22.

5.5 RTC Interrupt Block (RtcInterrupt)

The interrupt block contains the interrupt generation logic.

Figure 5-4 Raw and Masked Interrupt Generation Logic, illustrates the logic implementation used in generating the raw and masked interrupts.

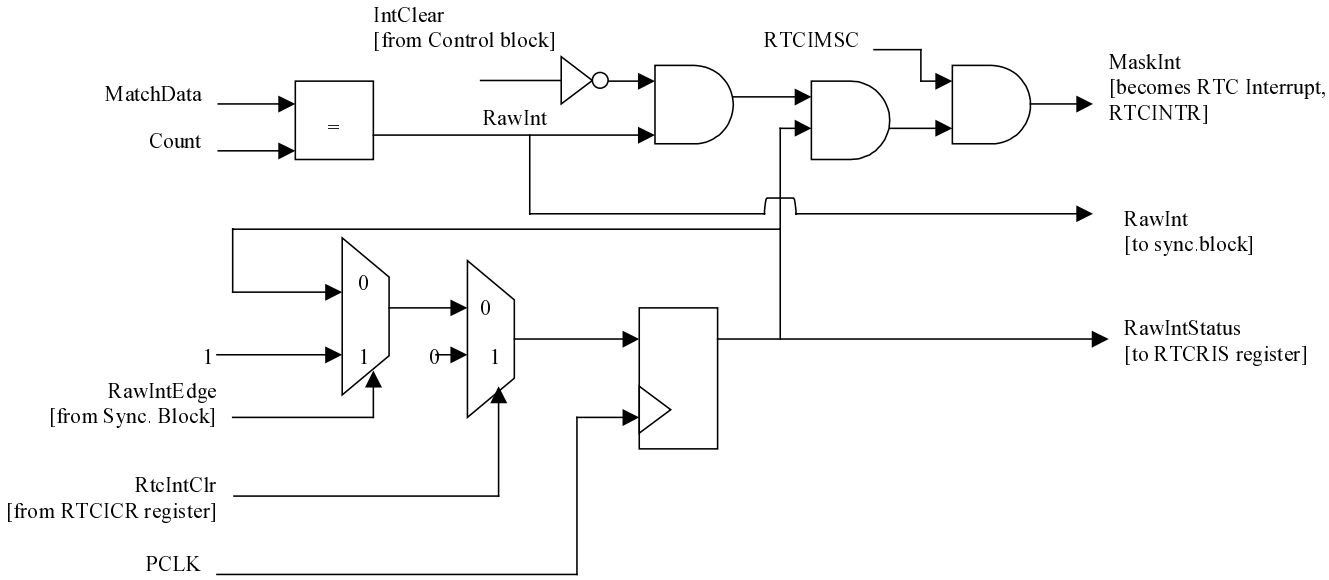


Figure 5-4 Raw and Masked Interrupt Generation Logic

The above interrupt generation logic is especially suited to clearing interrupts generated in slow asynchronous clock domains like the CLK1HZ domain.

The masked interrupt is asserted depending on the logic values of three signals and becomes the RTC interrupt signal, RTCINTR. It is also synchronised to PCLK to generate the masked interrupt status, RTCMIS. The signals are:

- **IntClear asserted LOW**. The IntClear signal ensures that once there is a write to the RTCICR (which asserts RtcIntClr) to clear the raw interrupt, the masked interrupt (MaskInt) is immediately de-asserted. This is necessary because once the raw interrupt is asserted it remains so until the next write to the match register, (RTCMR) or till the next rising edge of CLK1HZ – effectively a maximum latency of one second during which time the masked interrupt remains asserted.

  The IntClear signal is generated in the Control block.

- **RawIntStatus asserted HIGH**. This is the synchronised raw interrupt signal, which can be read from the raw interrupt status register, (RTCRIS). It is asserted when the RawIntEdge signal is HIGH and there hasn't been a write to RTCICR to clear the interrupt line. It ensures that MaskInt only remains asserted while there is a raw interrupt.

  The RawIntEdge signal is generated in the Synchronisation block. It goes HIGH when RawInt is asserted and synchronised to the PCLK domain.

- **RTCIMSC asserted HIGH**. If the interrupt mask is set, MaskInt is when there is a raw interrupt and if the previous two conditions have been satisfied.

The basic solution from which this variation is derived is the subject of UK Patent Application 0026941.5 [Ref.7].

5.6 RTC Synchronisation Block (RtcSynctoPCLK)

The synchronisation block handles the synchronisation of the following signals from the CLK1HZ domain to the PCLK domain.

- Count
- Raw interrupt, RawInt
- Masked interrupt, MaskInt

Synchronisation for the last two signals is achieved by passing the signals through two serial D-type flip-flops synchronous to the PCLK domain.

The multi-bit Count signal cannot be synchronised using this method since metastability may cause some bits of the vector to change in different clock cycles. The method used is first synchronise the LSB of the Count signal to the PCLK domain via two serial D-types and then to detect for a toggle of the LSB as this will change on every clock edge of the CLK1HZ signal. Immediately after which the multi-bit Count signal can be safely passed into the PCLK domain.

5.7 RTC Update block (RtcUpdate)

The Update block's primary function is to calculate the value of the real time clock (RTC). This value is read from the RTC data register, RTCDR. Unlike existing implementations where an initial or update value for the RTC is loaded directly into the counter and current or subsequent values are read directly from the counter; the value of the RTC is calculated in the update block in this particular implementation.

Other data generated in this block are the offset value and the equivalent match-register value (MATCHDATA). The offset value is the difference between the count value and the load register value. It is applied to the count and match-register values to generate an updated RTC value and MATCHDATA respectively.

The states in this state machine are:

- ST_IDLE. Idle State
- ST_OFFSET. State where offset value is calculated.
- ST_RTCVALUE. State where RTC value is calculated.
- ST_MATCHDATA. State where equivalent match-register value is calculated.

When the bus reset (PRESETn) is asserted, the state machine enters the ST_IDLE state and waits until the write enable signals for the RTC load register (WrEnRTCLR) or RTC match register (WrEnRTCMR) or the count increment indicator (CountEdge) has been asserted in order priority.

When WrEnRTCLR is asserted, it indicates that a new value has been written to the RTC load register and the state machine enters the ST_OFFSET state where a new offset value is calculated. Since the offset value is required to calculate the RTC value and MATCHDATA, the state machine unconditionally enters state ST_RTCVALUE and then ST_MATCHDATA if there isn't an immediate further write to the load register. The offset value is retained until a further write to the load register.

When WrEnRTCMR is asserted, it indicates that a new value has been written to the RTC match register and the state machine enters the ST_MATCHDATA state where a new equivalent match register value is calculated. If WrenRTCLR is asserted while in this state, the state machine enters the ST_OFFSET state to calculate a new offset value and thereafter unconditionally enters ST_RTCVALUE and then ST_MATCHDATA to update the outputs due to the change in the offset value.

When CountEdge is asserted, it indicates that there has been an increment of the Count register value and so the state machine enters the ST_RTCVALUE state where a new value of the RTC is calculated. If WrenRTCLR is

asserted while in this state, the state machine enters the ST_OFFSET state to calculate a new offset value and thereafter enters the remaining two states to update the outputs due to the change in the offset value.

Figure 5-5 shows the update logic state machine which cycles through the various states required in generating a new offset value, an updated value of the RTC and an equivalent match register value. And Figure 5-6 is a block diagram illustrating the implementation.

Note that MatchData is being asynchronously compared with the Count value and so does not require any synchronisation to the CLK1HZ domain.

Additionally, the match and offset registers are reset during a hard reset by the power-on reset signal, nPOR. This allows a correct read of the current RTC value even after a PCLK domain reset. It also allows an interrupt to be generated based on current RTC and MatchData values. The CLK1HZ domain reset signal should only be used during a hard reset, while the PCLK domain reset signal can be used during both a soft and hard reset.

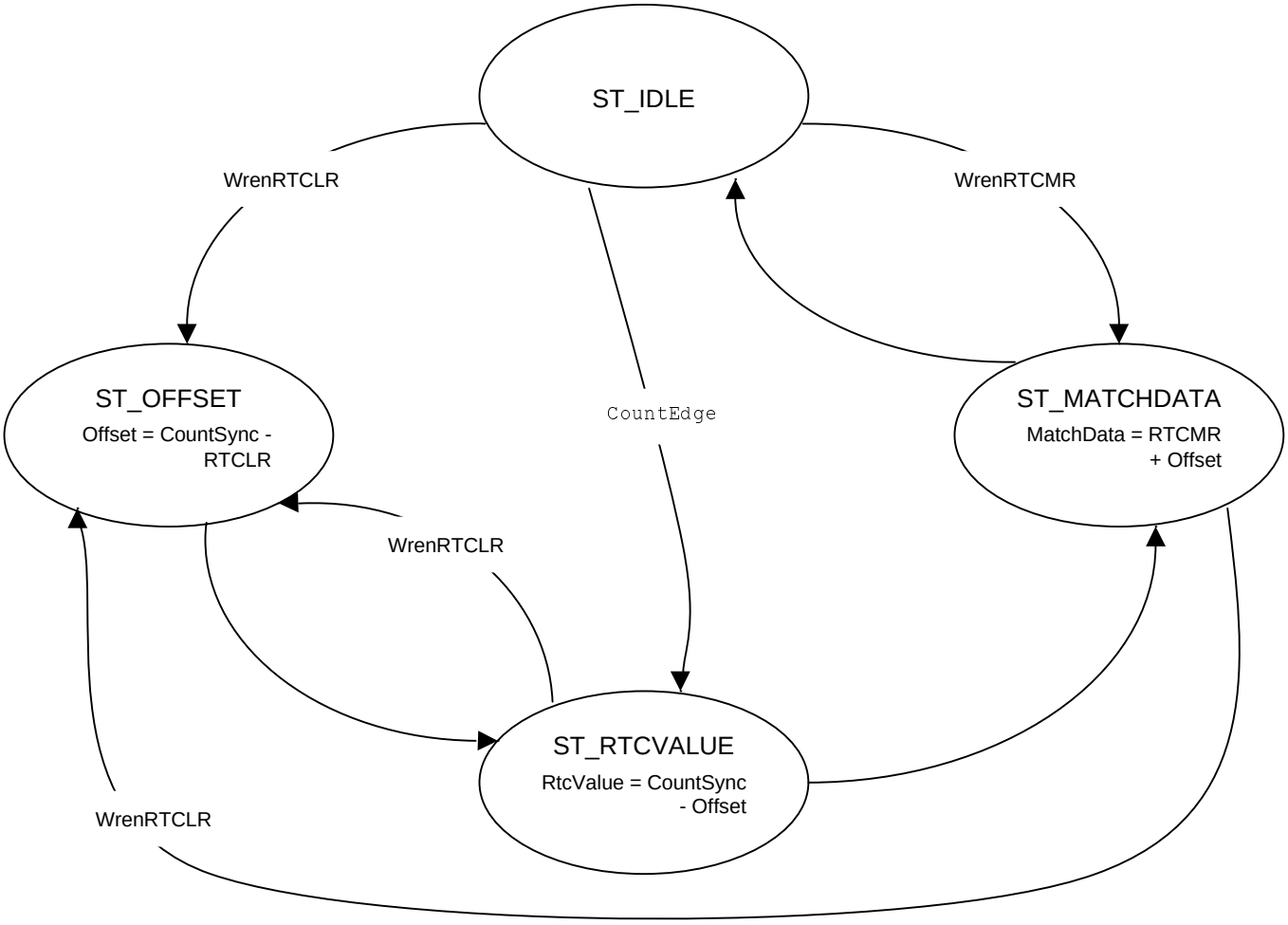


Figure 5-5 Update Logic state machine

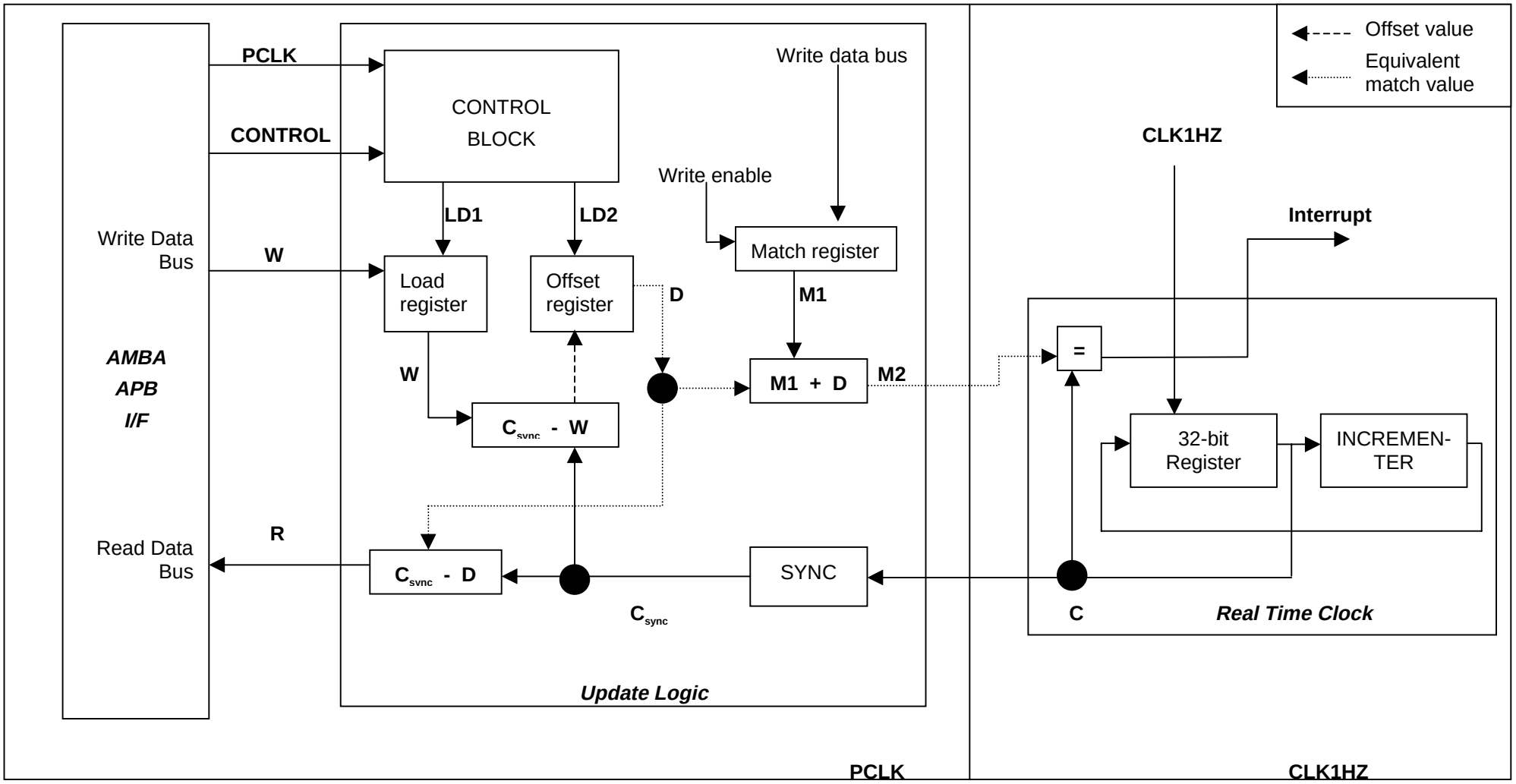


Figure 5-6 Interface mechanism with update logic between APB interface and Real Time Clock (RTC)

6 RTC FUNCTIONAL VERIFICATION DETAILS

The RTC Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the ***verification/vhdl*** and ***verification/verilog*** directories respectively. The test vectors that are applied to the BusTalk testbenches are in the ***verification/bustest*** directory.

6.1 RTC VHDL Verification Testbench

The ARM PrimeCell RTC VHDL test world uses a variant of a top-level testbench, which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref.3].

The testbench has been designed such that it can be used within most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the Unit Under Test (UUT) is given without the need to modify the testbench implementation. This transfer description takes the form of a text file that can be read by the specific testbench to which it is being targeted. Figure 6-1 illustrates the BusTalk VHDL testbench for running test vectors.

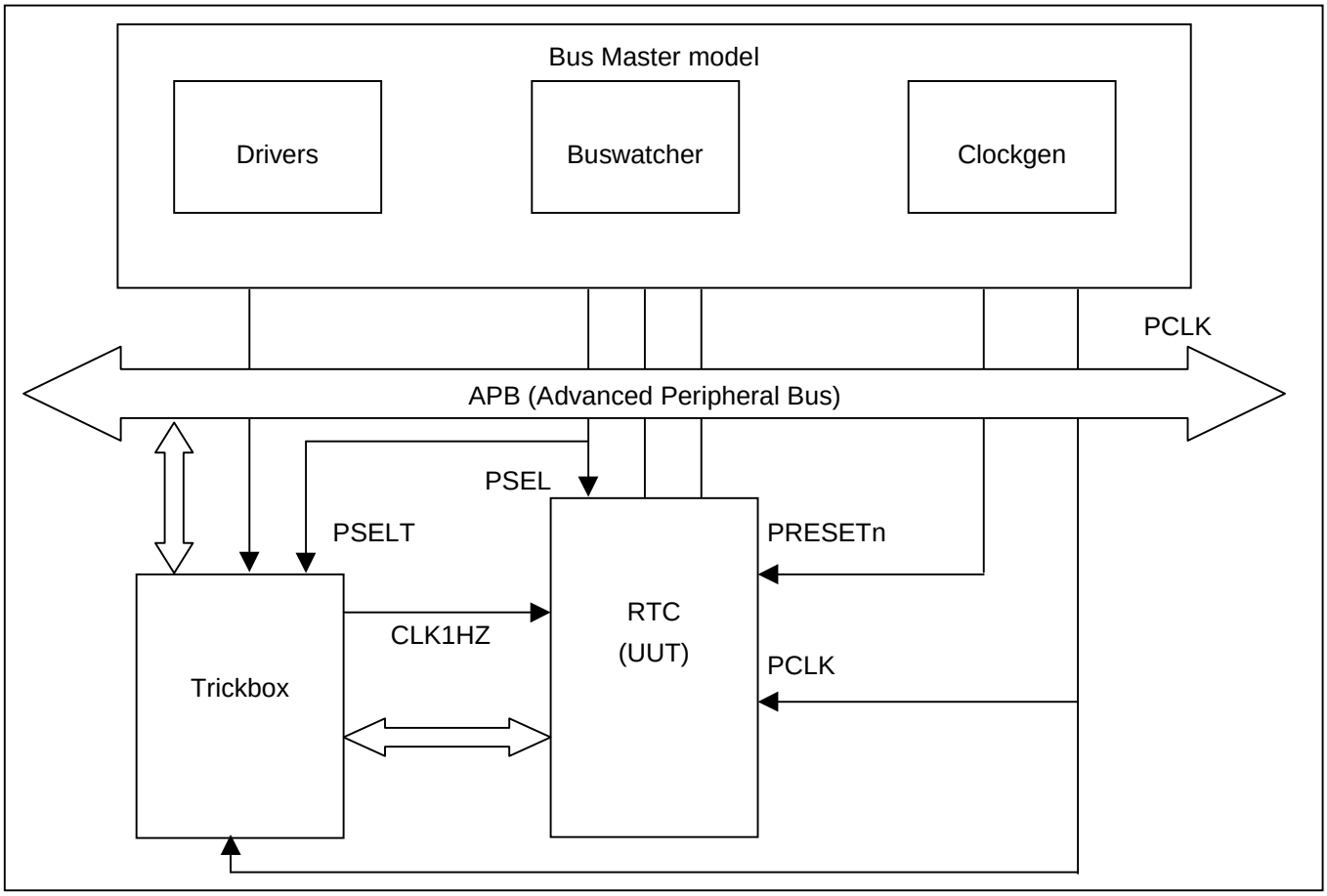


Figure 6-1 VHDL testbench for simulating test vectors

Figure 6-2 illustrates the hierarchy for the VHDL testbench.

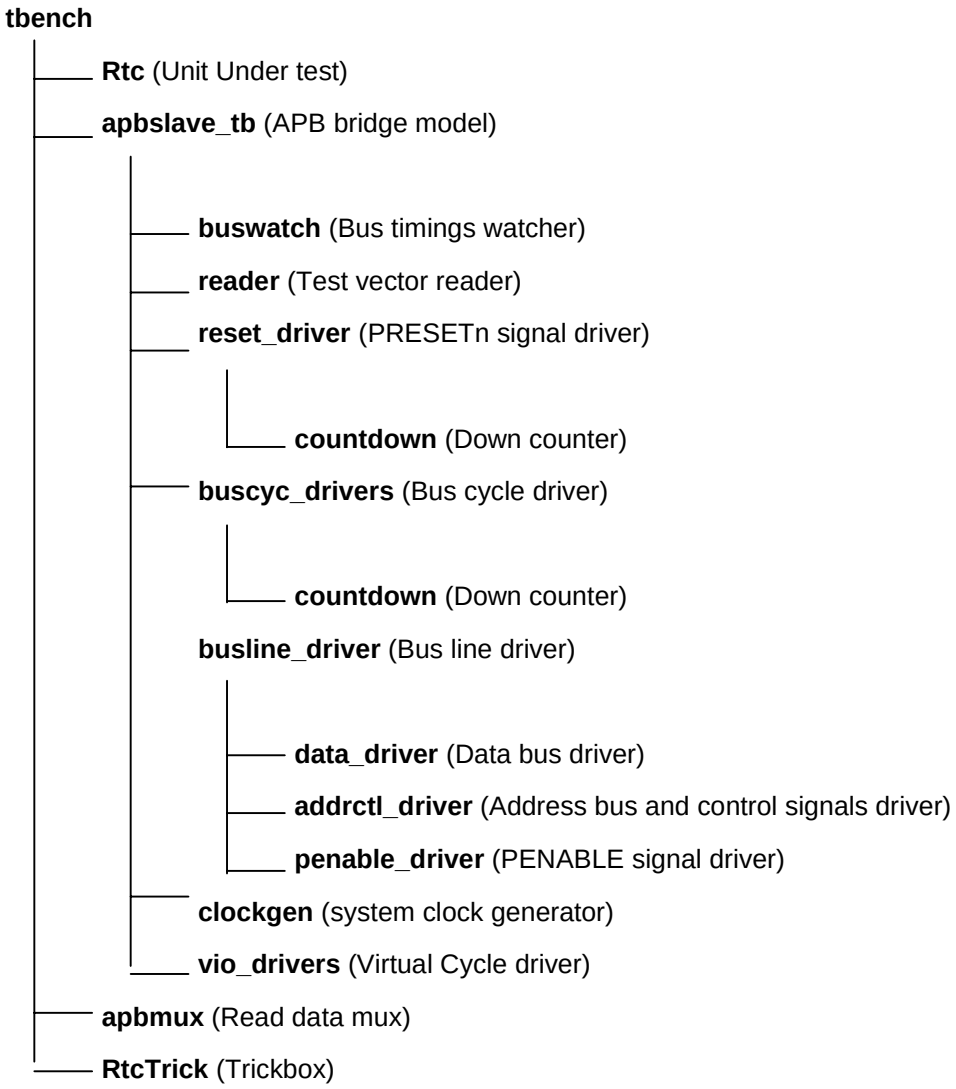


Figure 6-2 VHDL Testbench hierarchy

The testbench, ***tbench.vhd*** is used for functional simulations of the RTC. This module instantiates the RTC (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The RTC clock (CLK1HZ) is generated by the trickbox to facilitate frequency modifications from within the test code.

The CLK1HZ and its domain inputs to the RTC are driven by the trickbox. All the Scan inputs are tied low to keep them from interfering with the test process.

The default test frequency is set at 100MHz. This is the same frequency to which the RTC was constrained during synthesis. PCLK is set to the frequency defined in tbench.vhd using the tclkh and tclkl generic values when instantiating apbslave_tb.vhd.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- RTC VHDL RTL
- RTC VHDL netlist from VHDL RTL source

One of these versions may be referenced via the corresponding link to uut directory in **verification/vhdl**.

6.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the infile.bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. the busline_drivers drives the data bus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB Bridge model also provides the Select line for the trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

While the default behaviour of the bridge in the testbench is like a Rev E device, there is an option to force the bridge to issue commands with Rev D timings.

6.1.3 Trickbox

The functionality of the RTC is verified via a trickbox.

6.1.4 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.vhd file in the **verification/vhdl/tbench** directory. The buswatch module, which is in the verification/vhdl/buswatcher directory, checks the Read/Write databus timings and reports any violations in the timings.

6.1.5 APB Multiplexer

The read data buses from the RTC and trickbox are muxed together in this module before being connected to the APB Bridge model.

6.1.6 Test Vectors

The test vectors for the verification of the RTC are coded into separate C files depending on the logical region of the RTC which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The RTC testbench provides some configurability options using generics and constants. These generics and constants are configurable through the tbench.vhd file.

XonPSEL

XonPSEL is used in the tbench.vhd file that resides in **verification/vhdl/tbench**.

This generic is used to eliminate the 'X's that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.vhd the reset_driver.vhd and the buscyc_drivers.vhd which reside in **verification/vhdl/bid**.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.vhd, which resides in **verification/vhdl/bid**.

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd, which resides in **verification/vhdl/buwatcher.**

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If **SuppressOnReset** is TRUE, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SuppressOnReset** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

Mesg_enb

Mesg_enb is used in the buswatch.vhd, which resides in **verificationvhdl/buswatcher.**

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If **Mesg_enb** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches, which include more than one slave.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the RTC test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- verification/vhdl/Rtc_rtl/transcript

Run time logs for the netlist simulations are:

- verification/vhdl/Rtc_scaninsert_vhdl_net_max/transcript

Note: There are known Verilog-XL SDF annotation errors associated with LDV simulations for RTL and gate-level netlists. They do not affect the simulation results.

6.2 RTC Verilog Verification Testbench

The ARM PrimeCell RTC Verilog test world uses a variant of a top-level testbench, which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref.3].

The testbench has been designed such that it can be used within most common HDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the Unit Under Test (UUT) is given without the need to modify the testbench implementation. This transfer description takes the form of a text file that can be read by the specific testbench to which it is being targeted. Figure 1 illustrates the BusTalk Verilog testbench for running test vectors.

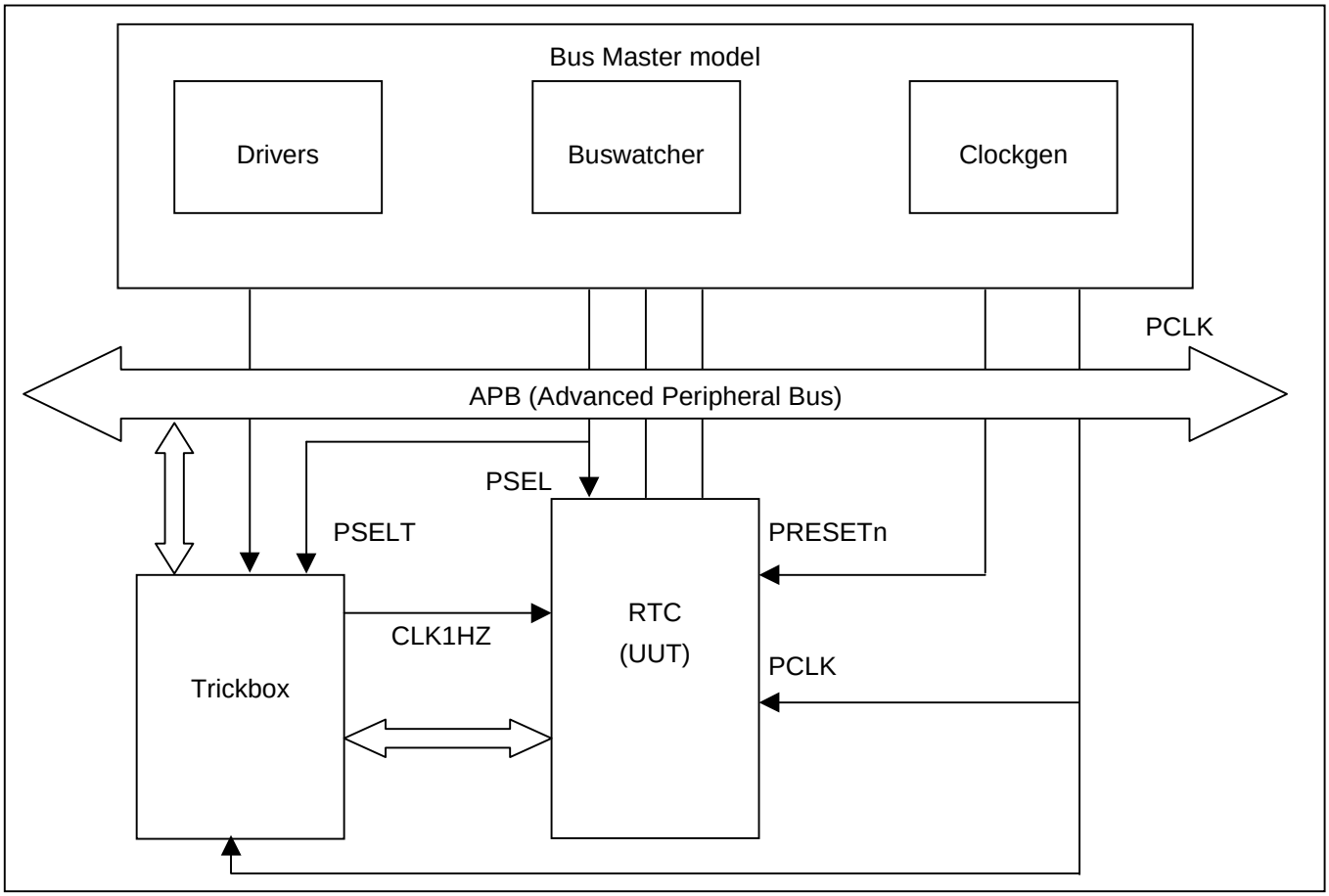


Figure 1 Verilog testbench for simulating test vectors

Figure 2 illustrates the hierarchy for the Verilog testbench.

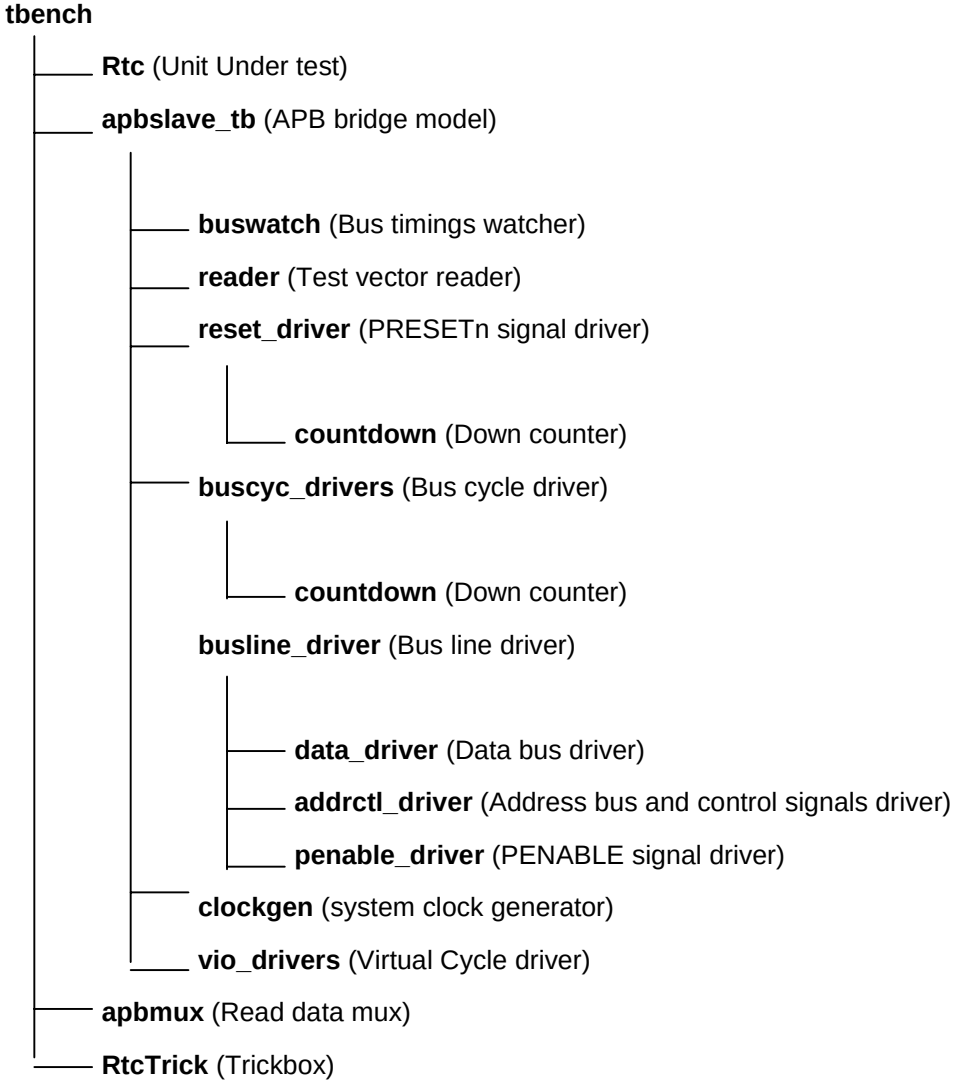


Figure 2 Verilog Testbench hierarchy

The testbench, ***tbench.v*** is used for functional simulations of the RTC. This module instantiates the RTC (Unit Under Test), the behavioural model of the APB Bridge, the trickbox to facilitate frequency modifications from within the test code.

The RTC clock (CLK1HZ) and its domain inputs to the RTC are driven by the trickbox. All the Scan inputs are tied low to keep them from interfering with the test process.

The default test frequency is set at 100MHz. This is the same frequency to which the RTC was constrained during synthesis. PCLK is set to the frequency defined in tbench.v using the tclkh and tclkl generic values when instantiating apbslave_tb.v in tbench.v. note that the ‘define values override the local parameters defined in clockgen.v in the ***verification/verilog/common*** directory. All the non-AMBA inputs to the RTC, except for the scan data inputs, are driven by the Trickbox.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- RTC Verilog RTL
- RTC Verilog netlist from Verilog source
- RTC Verilog netlist from VHDL source

One of these versions may be referenced via the corresponding link to uut directory in **verification/verilog** directory.

6.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the infile.bif file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. the busline_drivers drives the data bus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the trickbox.

6.2.3 Trickbox

The functionality of the RTC is verified via a trickbox.

6.2.4 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.v file in the **verification/verilog/tbench** directory. The buswatch module, which is in the ***verification/verilog/buswatcher*** directory, checks the Read/Write Databus timings and reports any violations in the timings.

6.2.5 APB Multiplexer

The read data buses from the RTC and trickbox are muxed together in this module before being connected to the APB Bridge model.

6.2.6 Test Vectors

The test vectors for the verification of the RTC are coded into separate C files depending on the logical region of the RTC, which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The RTC testbench provides some configurability options using parameters and defines. All the parameters, except MESG_ENB and SUPPRESSONRESET are configurable through the tbench.v file. MES_ENB and SUPPRESSONRESET are configurable through the buswatch.v that resides in **verification/verilog/buswatcher**. The define, DATA_CHECK is configurable through the apbmux.v that resides in **verification/verilog/tbench**.

XonPSEL

XonPSEL is used in the tbench.v file that resides in **verification/verilog/tbench**.

This define is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If **XonPSEL** is set to **0** in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to **1**, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.v the reset_driver.v and the buscyc_drivers.v which resides in **verification/verilog/bid**.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.v, which resides in **verification/verilog/bid**.

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESONRESET is used in the buswatch.v, which resides in **verification/verilog/buwatcher.**

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If **SUPPRESONRESET** is true the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If **SUPPRESONRESET** is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

MESG_ENB

MESG_ENB is used in the buswatch.v, which resides in **verification/verilog/buswatcher.**

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to FALSE. If **MESG_ENB** is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero

value when the UUT is not selected. This parameter will have to be set to FALSE in testbenches, which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the apbmux.v, which resides in **verification/verilog/tbench**.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When DATA_CHECK has a value **1**, these error messages are flagged. When it has a value **0**, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the RTC test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- verification/verilog/Rtc_rtl/Rtc_rtl_modelsim.log
- verification/verilog/Rtc_rtl/Rtc_rtl_LDV.log

Run time logs for the netlist simulations are:

- verification /verilog/Rtc_noscan_vhdl_net_min/Rtc_noscan_vhdl_net_min_modelsim.log
- verification/verilog/Rtc_scanready_verilog_net_typ/Rtc_scanready_verilog_net_typ_LDV.log

The code coverage results are:

- verification/verilog/Rtc_rtl/Rtc_rtl_modelsim_cover.log

Note: There are known Verilog-XL SDF annotation errors associated with LDV simulations for RTL and gate-level netlists. They do not affect the simulation results.

RTC Trickbox

The Trickbox is a programmable APB device designed to provide a test engineer a means to drive the non-APB input pins of the RTC and sample the non APB responses of the same.

6.2.9 Trickbox Signal Description

APB signal descriptions

Name	Type	Source / Destination	Description
PCLK	Input	From test bench	APB clock
PRESETn	Input	From test bench	Bus reset signal, active low
PADDR[7:2]	Input	From test bench	Subset of APB address bus
PRDATA[15:0]	Output	To test bench	Subset of unidirectional APB read data bus
PWDATA[15:0]	Input	From test bench	Subset of unidirectional APB write data bus
PSELT	Input	From test bench	Trickbox select signal from decoder. When set to 1 this signal indicates the slave device is selected and that a data transfer is required.
PENABLE	Input	From test bench	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	Input	From test bench	APB transfer direction signal indicates a write access when HIGH, read access when LOW.

RTC Data signals

Name	Type	Source / Destination	Description
CLK1HZ	Output	To RTC	RTC clock signal
nPOR	Output	To RTC	Power-on reset signal for offset and match registers.
nRTCRST	Output	To RTC	RTC reset signal for clocked elements in the CLK1HZ domain

RTC Interrupt lines

Name	Type	Source / Destination	Description
RTCINTR	Input	From RTC	RTC interrupt (active HIGH).

RTC Scan test line

Name	Type	Source / Destination	Description
SCANMODE	Output	To RTC	RTC scan test hold input. This signal must be asserted high during scan testing to ensure that internal data storage elements can be asynchronously reset. It must be driven LOW during normal use or when applying manufacturing test vectors.

Clock and Reset generation

The width of the high phase of the CLK1HZ signal is programmable through the RTCLK1HZH register and the width of the low phase of the CLK1HZ signal is programmable through the RTCLK1HZL register.

The RTC reset signal (nRTCRST) and power-on reset signal (nPOR) are generated from the system reset (PRESETn).

6.2.10 Trickbox functional description

The architectural behaviour of the Trickbox is similar to that of the RTC itself. The Trickbox contains the following major blocks:

APB Interface and the Register Block

The APB is a local secondary bus, which provides a low-power extension to the higher AHB within the AMBA system hierarchy. The APB interface generates read and write decodes for accesses to status/control registers. The register block stores data written or to be read across the APB interface.

Trickbox register summary

Address (offset from TrickboxBase)	Type	Width	Reset value	Name	Description
0x00	R/W	2	0x000	RTCR	Control Register
0x04	R	1	0x00	RTSR	Status Register
0x08	R/W	1	0x00	RTCLK1HZH	CLK1HZ High-phase register
0x0C	R/W	1	0x00	RTCLK1HZL	CLK1HZ Low-phase register
0x10	R	1	0x00	RTINTR	RTC Interrupt status register

6.3 Functional Verification Tests

6.3.1 Register Tests

Reset Value Tests

After a reset, each of the following registers is read to check if they contain the specified reset value.

Register Name	Reset Value	Register Name	Reset Value
RTCDR	0x00000000	RTCTOFFSET	0x00000000
RTCMR	0x00000000	RTCITOP	0x00000000
RTCLR	0x00000000	RTCPeriphID0	0x31
RTCCR	0x00000000	RTCPeriphID1	0x10
RTCIMSC	0x00000000	RTCPeriphID2	0x04
RTCRIS	0x00000000	RTCPeriphID3	0x00
RTCMIS	0x00000000	RTCPCellID0	0x0D
RTCICR	0x00000000	RTCPCellID1	0xF0
RTCITCR	0x00000000	RTCPCellID2	0x05
RTCTCOUNT	0x00000000	RTCPCellID3	0xB1

Register Read/Write Tests

A data pattern is written to each of the following read/write registers:

- RTCMR
- RTCLR
- RTCCR
- RTCIMSC
- RTCTCR
- RTCTOFFSET
- RTCTCOUNT

The register contents are read back to see if it contains the written value. if the values are the same then the register is declared to be writeable and readable and the test passes.

6.3.2 Counter Test

This test checks for the correct operation of the counter. An initial value is written to the test count register (RTCTCOUNT) and then the register is read back after a known number of CLK1HZ cycles to determine if the counter increments correctly.

In addition, the operation of the counter near the maximum value of FFFFFFFF is checked to see if it wraps around correctly to 00000000 and continues incrementing.

6.3.3 Update Test

This test checks for the correct operation of the Update state machine, which generates:

- The Offset value
- The value of the RTC
- The equivalent match-register value.

Various values are written to the RTC load register (RTCLR) to initiate the operation of the state machine. The data is read back from the test offset register to verify that the correct offset value was calculated. Data is also read back from the RTC data register to verify that the counter increments correctly after ever CLK1HZ rising edge.

6.3.4 Interrupt Test

This test checks for the generation of the RTC interrupt (RTCINTR).

First the RTC interrupt mask set/clear register (RTCIMSC) is set to enable interrupts and then consecutive values are written to the RTC load and match register.

After the next rising edge of **CLK1HZ** the status of the interrupt is checked via the RTC masked status register (RTCMIS) to confirm that the interrupt has been asserted.

Afterwards, the interrupt is cleared by writing to the RTC interrupt clear register (RTCICR) and again the RTCRIS register is checked to confirm that the interrupt has been cleared.

7 RTC INTEGRATION TEST DETAILS

The RTC integration test vector suite allows the integrator to verify that the RTC has been connected into the target system correctly. These vectors test 3 classes of signals:

a) AMBA signals (RTC signals connected directly to the APB)

b) Intra-Chip signals (RTC signals connected to the Interrupt controllers)

c) Primary I/O signals (normally for signals connected to the chip pads but none in this case)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7]. The RTC integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the RTC has been connected into the target system correctly.

7.1 RTC VHDL Integration testbench

The AMBA TICTalk testbench used for integration testing of the RTC is located in the integration/vhdl/testbench directory. The Integration test vectors are located in the integration/bustest directory.

The test vectors used with this testbench can be applied to the RTC when it is part of an AMBA system design. Figure 7-1 illustrates the testbench to apply TICTalk test vectors.

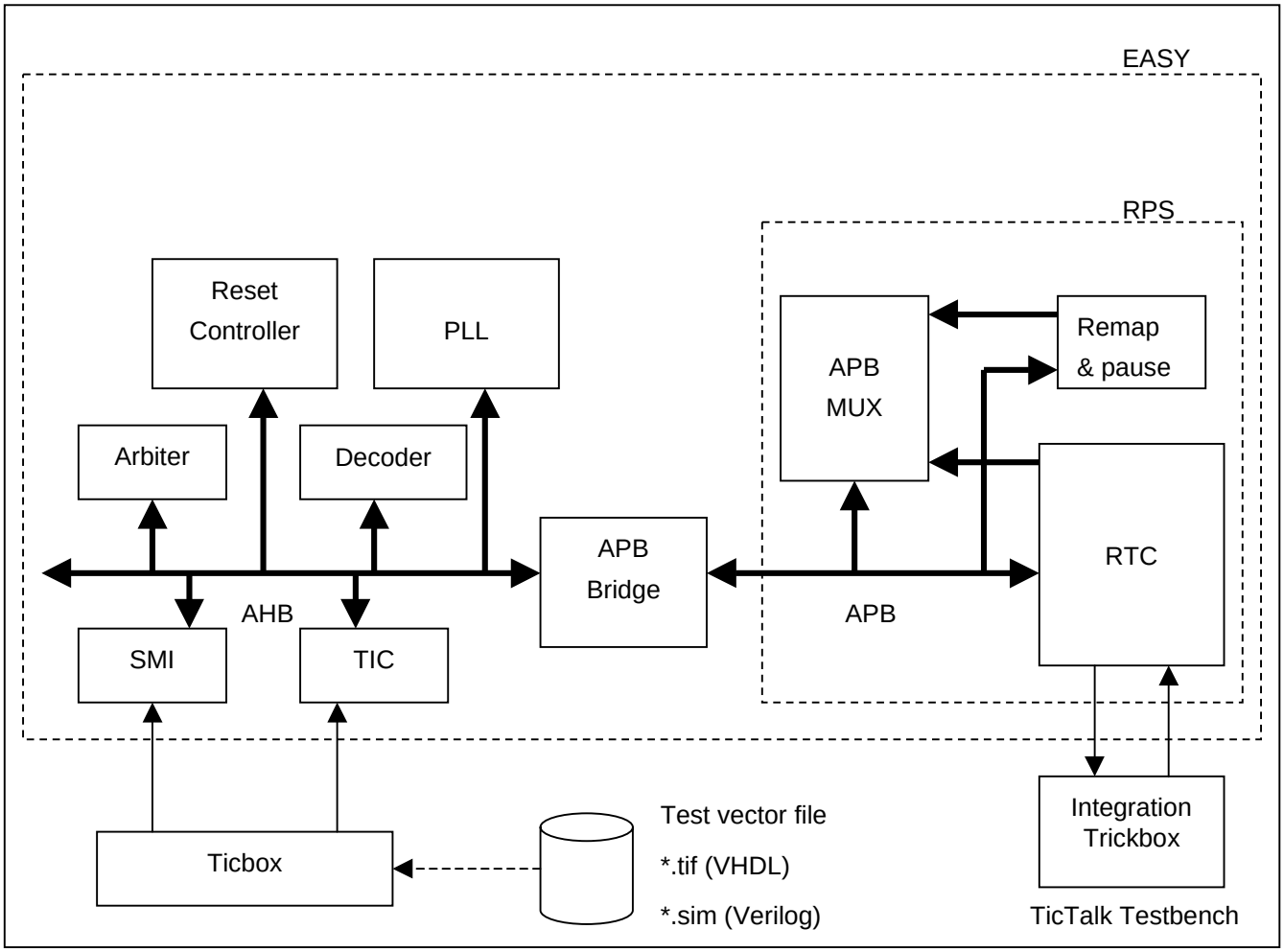


Figure 7-1 Testbench for simulating TICTalk test vectors

The RTC Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7-2 Testbench hierarchy for simulating TICTalk test vectors illustrates the hierarchy for the TICTalk testbench.

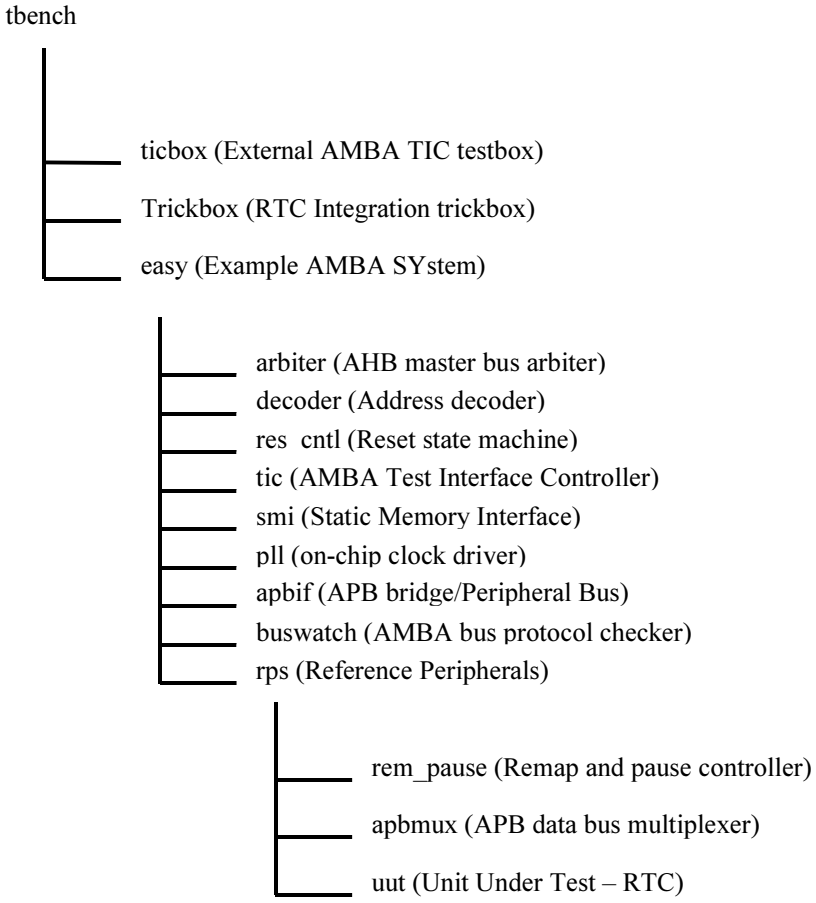


Figure 7-2 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.vhd**, the top-level of the testbench that is used for running the integration test of the RTC. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The RTC is instantiated in the **rps.vhd**, which is part of the EASY system. The CLK1HZ is generated in the rps.vhd. All the scan inputs are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.1.1 Unit Under Test

The unit under test may be one of the following:

- RTC VHDL RTL
- RTC VHDL netlist from VHDL RTL Source

One of these two versions may be referenced via the corresponding link to the uut directory in integration/vhdl .

7.1.2 Test Vectors

The test vectors for the integration testing of the RTC are coded into separate C files for testing the reset values of all the RTC registers and for Integration testing. Make options are provided to run the test together or individually.

7.1.3 Running Simulations

The README file in the **integration/vhdl/tbench** directory gives step by step instructions for running simulations using the RTC Integration test vectors on the VHDL source code.

Run time logs are available in the following files:

integration/vhdl/Rtc_rtl/Rtc_rtl_modelsim.log

integration/vhdl/Rtc_scanready_vhdl_net_max/Rtc_scanready_vhdl_net_max_modelsim.log

7.2 RTC Verilog Integration testbench

The test vectors used with this testbench can be applied to the RTC when it is part of an AMBA system design. Figure 7-3 illustrates the testbench to apply TICTalk test vectors.

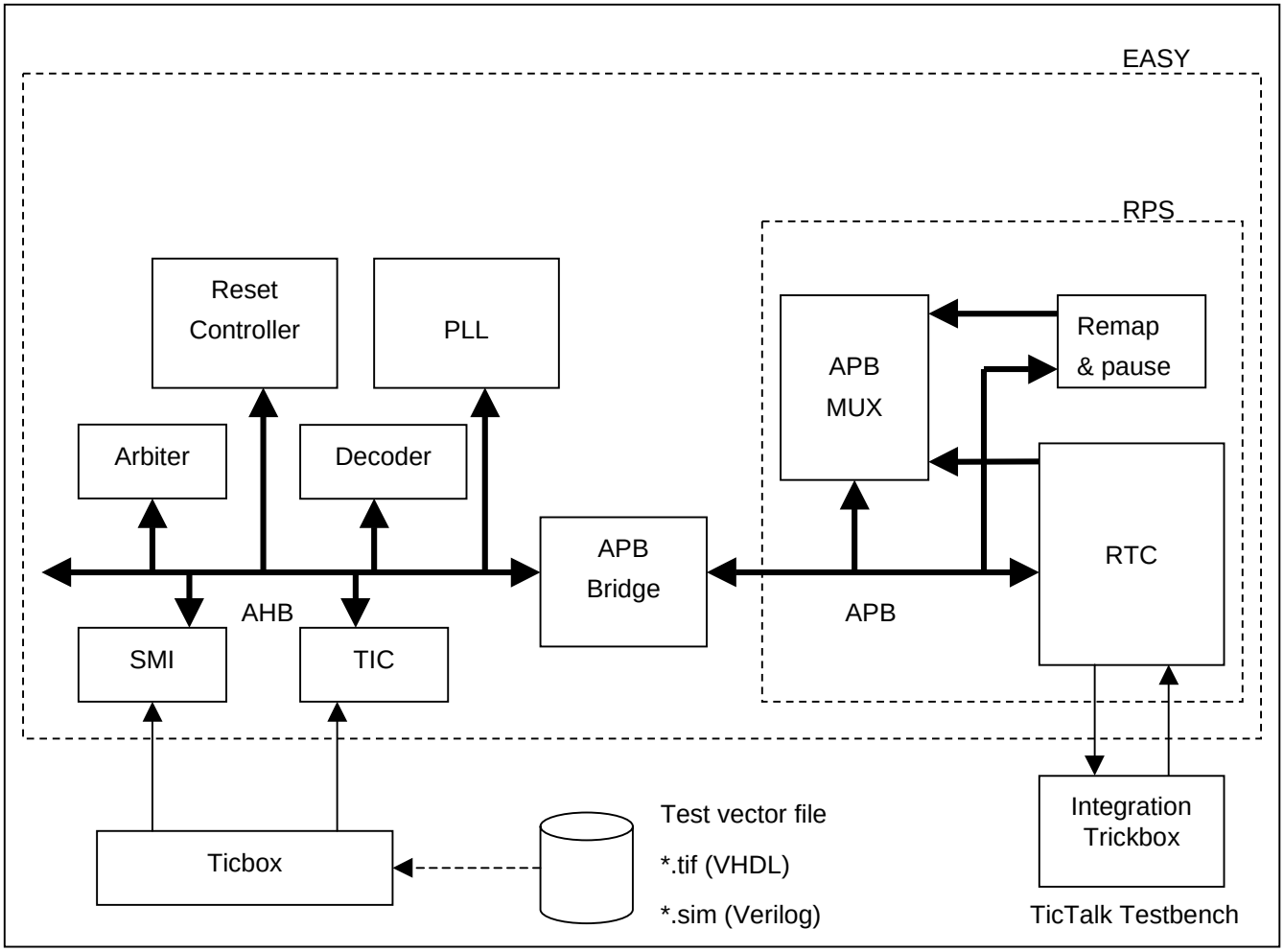


Figure 7-3 Testbench for simulating TICTalk test vectors

The RTC Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 7-4 Testbench hierarchy for simulating TICTalk test vectors illustrates the hierarchy for the TICTalk testbench.

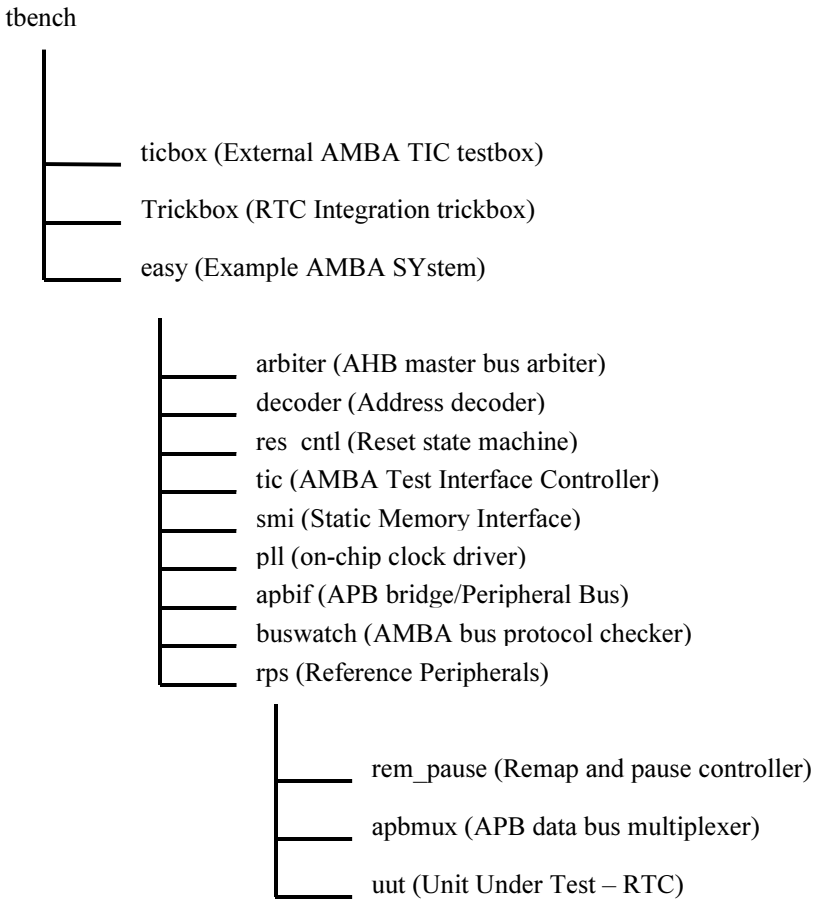


Figure 7-4 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to **tbench.v**, the top-level of the testbench that is used for running the integration test of the RTC. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The RTC is instantiated in the **rps.v**, which is part of the EASY system. The CLK1HZ is generated in the **rps.v**. All the scan inputs are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.2.1 Unit Under Test

The unit under test may be one of the following:

- RTC Verilog RTL
- RTC Verilog netlist from VHDL RTL Source
- RTC Verilog netlist from Verilog RTL Source

One of these two versions may be referenced via the corresponding link to the uut directory in integration/verilog.

7.2.2 Test Vectors

The test vectors for the integration testing of the RTC are coded into separate C files for testing the reset values of all the RTC registers and for Integration testing. Make options are provided to run the test together or individually.

7.2.3 Running Simulations

The README file in the **integration/verilog/tbench** directory gives step by step instructions for running simulations using the RTC Integration test vectors on the VHDL source code.

Run time logs are available in the following files:

integration/verilog/Rtc_rtl/Rtc_rtl_LDV.log

integration/verilog/Rtc_rtl/Rtc_rtl_modelsim.log

integration/verilog/Rtc_noscan_vhdl_net_min/Rtc_noscan_vhdl_net_min_modelsim.log

integration/verilog/Rtc_scanready_verilog_net_typ/Rtc_scanready_verilog_net_typ_LDV.log

7.3 Integration Tests

7.3.1 Integration Testing of Intra-Chip outputs

Please refer back to Section 5.3.4on Page 19 as it covers the test logic and test sequences.